

```
--- djangoapi\core\myLib\manageUsers.py ---
```

```
from django.contrib.auth.models import User
from typing import Union, List
```

```
def createUser(username:str, email:str, password: str, is_active: bool=True,
is_superuser:bool=False)->User:
    u=User(username=username, email=email)
    u.set_password(password)
    u.is_active=is_active
    u.is_superuser=is_superuser
    u.save()
    return u
```

```
def getUserGroups(user: User):
    """
    Gets a lists with the user groups that the user belongs. The user is an object of
    the
        django.contrib.auth.models.User class
    """
    l = user.groups.values_list('name',flat = True) # QuerySet Object
    return list(l)
```

```
def getUserGroups_fromUsername(username):
    """
    Gets a lists with the user groups that the user belongs. The username is the
    username,
    usually an email
    """
    user=User.objects.get(username=username)
    return getUserGroups(user)
```

```
def getUserGroupsAsDict(user_username_or_id: Union[User, str, int]):
    try:
        # Detectar si el identificador es un entero (ID) o una cadena (username)
        if isinstance(user_username_or_id, int):
            user = User.objects.get(id=user_username_or_id)
        elif isinstance(user_username_or_id, str):
            user = User.objects.get(username=user_username_or_id)
        elif isinstance(user_username_or_id, User):
            user = user_username_or_id
        else:
            raise ValueError("El identificador debe ser un ID (int) o un nombre de
            usuario (str).")

        # Obtener los grupos del usuario
        groups = user.groups.values('id', 'name')
        return list(groups)

    except User.DoesNotExist:
        return []
    except ValueError as e:
```

```
return {"error": str(e)}
```